

TRABAJO PRÁCTICO 7: Comunicación con MongoDB (Drivers y ODM)

Estudiante: Damian Tristant

Materia: Base de Datos 2

Parte 1: Implementación con Driver Nativo

1) Inicialización e instalación:

```
npm init -y
```

```
npm install mongodb
```

2) Archivo driver_lab.js:

Copio el texto plano ya que la entrega no requería capturas en esta ocasión.

```
const { MongoClient } = require('mongodb');
```

```
// Cadena de conexión local o de Atlas
```

```
const url = 'mongodb://localhost:27017';
```

```
const client = new MongoClient(url);
```

```
const dbName = 'laboratorio';
```

```
async function main() {  
  try {  
    await client.connect();  
    console.log('Conectado exitosamente al servidor de  
base de datos');  
  
    const db = client.db(dbName);  
    const collection = db.collection('sensores');  
  
    // Inserción de los tres sensores de robótica  
    const sensoresPrueba = [  
      { nombre: 'Sensor de Humedad', tipo: 'Analógico',  
estado: 'Activo' },  
      { nombre: 'Servo Motor', tipo: 'Actuador', estado:  
'Activo' },  
      { nombre: 'LED RGB', tipo: 'Diodo', estado: 'Inactivo' }  
    ];  
  
    const insertResult = await  
collection.insertMany(sensoresPrueba);
```

```
    console.log(` ¡Éxito! Se insertaron  
    ${insertResult.insertedCount} sensores.` );  
  
  } catch (error) {  
    console.error('Error durante la ejecución:', error);  
  } finally {  
    await client.close();  
  }  
}  
  
main();
```

Parte 2: Implementación con Mongoose (ODM)

1) Instalación del paquete:

```
npm install mongoose
```

2) Archivo mongoose_lab.js:

```
const mongoose = require('mongoose');
```

```
// Conexión a la base de datos laboratorio usando
mongoose

mongoose.connect('mongodb://localhost:27017/laboratorio')

  .then(() => console.log('Conectado a MongoDB
mediante Mongoose...'))

  .catch(err => console.error('No se pudo conectar a
MongoDB...!', err));
```

```
// Definición del Esquema con validaciones y
timestamps

const componenteSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true
  },
  stock: {
    type: Number,
    default: 0
  }
}, { timestamps: true }); // Activa createdAt y updatedAt
automáticamente
```

```
// Creación del Modelo
```

```
const Componente = mongoose.model('Componente',  
componenteSchema);
```

```
async function probarInsercion() {
```

```
  try {
```

```
    // Inserción de prueba
```

```
    const nuevoComponente = new Componente({
```

```
      nombre: 'Placa Arduino Uno',
```

```
      stock: 25
```

```
    });
```

```
    const resultado = await nuevoComponente.save();
```

```
    console.log('Componente de prueba insertado con  
éxito:', resultado);
```

```
  } catch (error) {
```

```
    console.error('Error al insertar el componente:',  
error);
```

```
  } finally {
```

```
    mongoose.connection.close();
```

```
}
```

```
}
```

```
probarInsercion();
```

Parte 3: Reflexión Final

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?

La función principal del driver es actuar como un puente de comunicación. Traduce las instrucciones, consultas y comandos escritos en el lenguaje de programación de la aplicación (como funciones asincrónicas u objetos en JavaScript) al protocolo de red y formato de comandos específicos que entiende el motor de MongoDB, permitiendo enviar y recibir datos de manera transparente.

2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?

El driver convierte los objetos nativos de JavaScript al formato BSON antes de enviarlos al servidor de MongoDB. BSON es la representación binaria de JSON

que utiliza el motor para almacenar y procesar documentos de manera súper eficiente y rápida.

3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?

Elegiría usar el driver nativo en proyectos donde el rendimiento extremo, la baja latencia y el procesamiento de altos volúmenes de datos sean críticos (como la ingesta masiva de datos en tiempo real de sensores IoT o procesos de analítica pesada de fondo). Al no tener las capas intermedias de abstracción, validación de esquemas y ganchos de ciclo de vida que agrega Mongoose, el driver nativo procesa las operaciones CRUD con el máximo control y velocidad posibles directamente sobre el motor.